

Illustration:

Here, we illustrate how the algorithm works.

1. You obtain (declare) an array Tab[100] of integer.
2. Suppose the maximum value of the range given is N=10
3. Fill array Tab with N numbers.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

4. Set entry 1 (the first) to zero.

0	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Now, we identify all the prime numbers from 2 to rootN and set to zero their multiples found in the array as follows:

5. Round up the square root of N (i.e, of 10) to get 4.
6. Identify 2 as a prime number
7. Excluding 2 itself, we set all multiples of 2, right up to N. So, we reset the numbers 4, 6, 8 and 10 to obtain:

0	2	3	0	5	0	7	0	9	0
---	---	---	---	---	---	---	---	---	---

8. Identify the next prime number (i.e. 3)
9. Again, executing 3, we set all the attributes of 3 (i.e, 3 and 9) to zero to obtain:

0	2	3	0	5	0	7	0	0	0
---	---	---	---	---	---	---	---	---	---

10. The next available [non-zero] number, 5, is greater than rootN (i.e, 4) so, we stop. We now display (or print) all the non-zero numbers of the array as the prime numbers found between 1 and 10. Viz: 2, 3, 5 and 7.

Application:

Here, we develop the required program by following the steps below:

- (i) In your answer booklet, state the programming language you will use. Then, give a more detailed version of the algorithm above in pseudocode. In it, let N=20 (3marks)

```
1. Declare an array Tab[100] of integers.
2. Set N = 20.
3. Fill array Tab with numbers from 1 to N.
4. Set Tab[1] = 0.
5. Set rootN = ceil(sqrt(N)).
6. For i from 2 to rootN:
    a. If Tab[i] is not 0:
        i. Set j = i^2.
        ii. While j <= N:
            A. Set Tab[j] = 0.
            B. Increment j by i.
7. Print all non-zero numbers in the array Tab, which represent prime numbers between 1 and N.
```

- (ii) Write a subprogram called initialize(tab, n) that fill the array tab with numbers from 1 to n (4marks)

```
void initialize(int tab[], int n) {
    int i;
    for (i = 1; i <= n; i++) {
        tab[i] = i;
    }
}
```

- (iii) Write a subprogram called verifyprime(x) that takes a number x and verifies whether it is a prime number or not. For this, x is not divisible by any non-zero number less than it in the array tab. (5marks)

```
int tab[100]; // Declare tab as a global variable
bool verifyprime(int x) {
    int i;
    for (i = 2; i < x; i++) {
        if (tab[i] != 0 && x % tab[i] == 0) {
            return false; // x is divisible by a non-zero number less than it
        }
    }
    return true; // x is not divisible by any non-zero number less than it
}
```

- (iv) Write a subprogram called setzero(tab, n, y) that takes a number y and sets its multiples in array tab to zero, except the number itself, as explained earlier (4marks)

```
void setzero( int tab[], int n, int y) {
    int i;
    for (i = 2 * y; i <= n; i += y) {
        tab[i] = 0;
    }
}
```

- (v) Write a subprogram called doDisplay(tab, n) that displays all the non-zero elements of array tab, up to those in array position n. (3marks)

```
void doDisplay(int tab[], int n) {
    int i;
    printf("Prime numbers: ");
    for (i = 2; i <= n; i++) {
        if (tab[i] != 0) {
            printf("%d ", tab[i]);
        }
    }
}
```

- (vi) Write a main program that makes use of the above subprograms to identify prime numbers in a positive integers range. Make sure your program works correctly. (4marks)

Final code with main program

```
#include<stdio.h>
#include<conio.h>
#include <stdbool.h>
#include <math.h>
//By Mr. Lawrence 670626302
int tab[200]; // Declare tab as a global variable
void initialize(int n) {
    int i;
    for ( i = 1; i <= n; i++) {
        tab[i] = i;
    }
}

void setzero( int n, int y) {
    int i;
    for (i = 2 * y; i <= n; i += y) {
        tab[i] = 0;
    }
}

void doDisplay(int n) {
    int i;
    printf("Prime numbers: ");
    for ( i = 2; i <= n; i++) {
        if (tab[i] != 0) {
            printf("%d ", tab[i]);
        }
    }
    printf("\n");
}

bool verifyprime(int x) {
    int i;
    for ( i = 2; i <= sqrt(x); i++) {
        if (x % i == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int i , N;
```

```

printf("enter a number to Test your program: ");
scanf("%d",&N);

// Call the initialize function
initialize(N);

for (i = 2; i <= sqrt(N); i++) {
    if (tab[i] != 0) {
        setzero(N, i);
    }
}

// Display prime numbers
doDisplay(N);
getch();
return 0;
}

```

- (vii) Print your entire program including its subprograms (1mark)
 (viii) Test your program with N=100. Save its output and then print it (2marks)

The screenshot shows a C program editor window titled '2022ALCSC.c' with the following code:

```

#include<stdio.h>
#include<conio.h>
#include <stdbool.h>
#include <math.h>
//By Mr. Lawrence 670626302

```

Below the editor is a terminal window showing the execution of the program 'D:\Secondary School\C Program Tasks\2022ALCSC.exe'. The user enters '100' when prompted, and the program outputs the prime numbers up to 100: 'Prime numbers: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97'.

Explanation:

Certainly! Here's an explanation of the program and its subprograms:

1. *initialize(int n)*:

- This subprogram fills the array tab with numbers from 1 to n.
- It iterates from 1 to n and assigns each array element with its corresponding index value.

2. *setzero(int n, int y)*:

- This subprogram sets multiples of a given number y in the array tab to zero, except the number itself.

- It starts from $2 * y$ (the first multiple of y greater than y) and increments by y , setting each multiple to zero until reaching n .

3. ***doDisplay(int n)*:**

- This subprogram displays all the non-zero elements of the array `tab` up to the position n .
- It iterates through the array and prints each non-zero element.

4. ***verifyprime(int x)*:**

- This subprogram verifies whether a given number x is a prime number or not.
- It checks whether x is divisible by any number from 2 up to the square root of x . If it is divisible, it returns false; otherwise, it returns true.

5. ***Main program*:**

- It initializes the array `tab` with numbers from 1 to 100.
- It applies the Sieve of Eratosthenes algorithm by iterating through numbers from 2 to the square root of 100 and calling the `setzero` function to mark multiples of each prime number as zero.
- Finally, it calls the `doDisplay` function to display all the prime numbers found in the range.

This program efficiently identifies prime numbers within the range specified by N . Let's test it with $N = 100$ to verify its correctness.
